



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

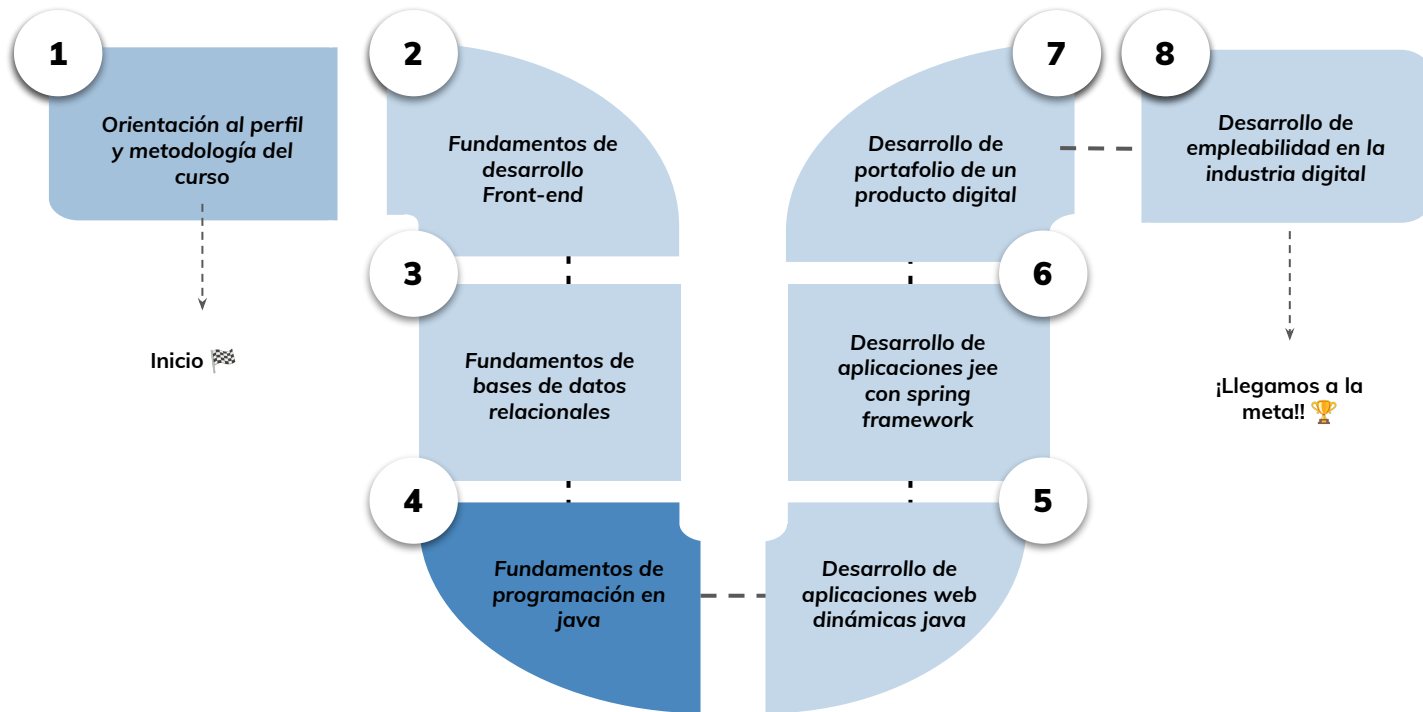


› Desarrollo de aplicaciones de consola en Java - Parte 2

Aprendizaje Esperado: Implementar una aplicación básica de consola utilizando las buenas prácticas y convenciones para resolver un problema de baja complejidad acorde al lenguaje Java.

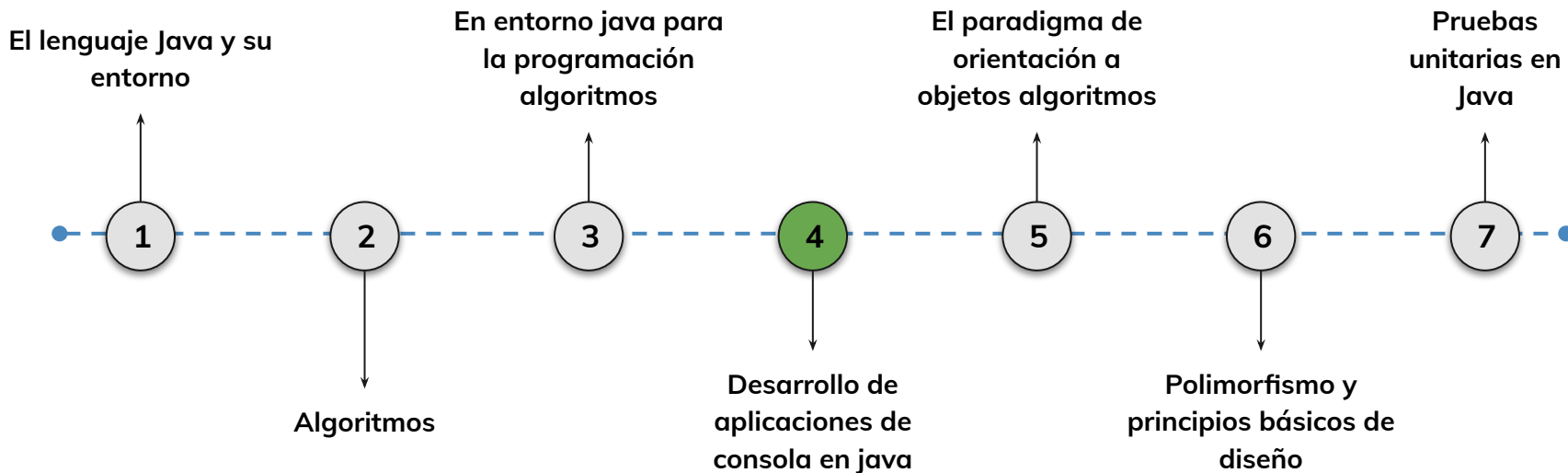
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

4.

Documentación y mantenimiento de aplicaciones de consola en Java

Aprenderás a utilizar *JavaDoc* para documentar clases, métodos y estructuras en Java. Además, aplicarás buenas prácticas de codificación para hacer el código más claro, mantenible y profesional.

JavaDoc y documentación profesional

Sintaxis y etiquetas

Generación de documentación HTML

Buenas prácticas de codificación y depuración

Estándares de nombres, formato y organización

Técnicas de depuración en Eclipse IDE



Objetivos de aprendizaje

¿Qué aprenderás?



- Utilizar JavaDoc para documentar clases, métodos y parámetros.
- Aplicar etiquetas de documentación como @param, @return, @author.
- Generar documentación navegable en HTML desde Eclipse.
- Corregir código mal formateado siguiendo convenciones.
- Detectar y depurar errores utilizando breakpoints y el modo Debug.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

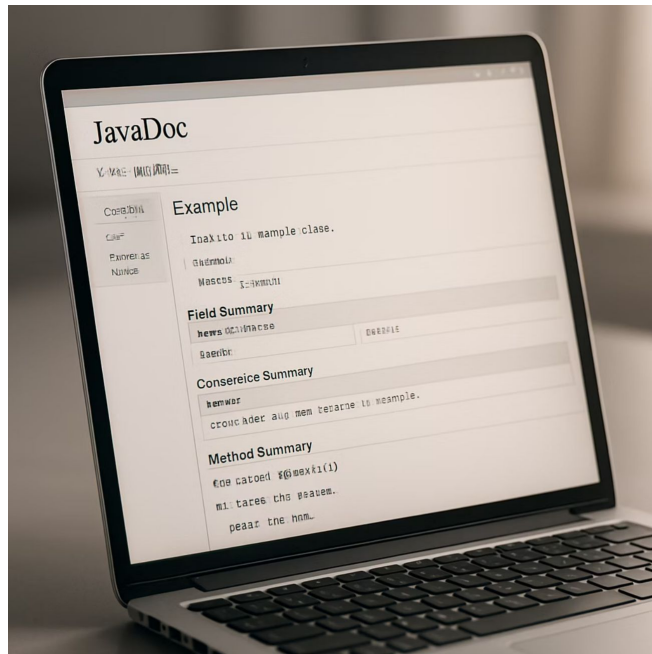
- Estándares de codificación en Java (nombres, organización y estilo)
- Creación de aplicaciones de consola usando Scanner
- Exportación de la aplicación como archivo .jar
- Uso de Eclipse IDE para depuración paso a paso

Desarrollo de aplicaciones de consola en java

Documentando el código con Javadoc

La documentación en Java se realiza utilizando Javadoc, una herramienta que genera documentación en formato HTML a partir de comentarios especiales escritos en el código fuente. Los comentarios Javadoc se agregan encima de declaraciones de clases, métodos, campos y paquetes, y proporcionan información sobre su propósito, funcionamiento y uso.

Debemos recordar que el código bien documentado es un código mucho más legible y mantenible.



```
/**  
public static int sumar(int a, int b) {  
    return a + b;  
}
```

```
/**  
 *  
 * @param a  
 * @param b  
 * @return  
 */  
public static int sumar(int a, int b) {  
    return a + b;  
}
```

¿Cómo se comenta?

Los comentarios de documentación se inician con `/**` y se cierran con `*/`. Pero simplemente colocando `/**` o `/*` y dando "Enter", nos generará la plantilla para documentar.

A continuación te mostraremos las distintas posibilidades para documentar nuestro código

Documentando una Clase

Cuando documentamos una clase en JavaDoc, proporcionamos una descripción general de su propósito, su funcionalidad y su uso. Es fundamental incluir información sobre los campos, constructores y métodos que la componen, explicando su comportamiento y las relaciones entre ellos.

Además de una descripción clara, es una buena práctica incluir el autor, la versión y cualquier nota importante sobre la clase. Esto ayuda a otros desarrolladores a entender rápidamente cómo interactuar con ella y cómo se espera que funcione dentro del sistema, mejorando la mantenibilidad y legibilidad del código a largo plazo.

```
1 package main.java;
2
3 /**
4  * Descripción de la clase.
5  */
6 public class Logica {
7
8     /**
9      * Descripción de la variable
10     */
11     private MiEnum variableEnum;
12
13     public Logica() {
14         // TODO Auto-generated constructor stub
15     }
16
17     /**
18      * Descripción de la función.
19      * @param a sera una variable entera
20      * @param b sera una variable entera
21      * @return Esta función devolverá un valor de la división de a y b
22      * @throws ArithmeticException posible error que lance la función
23     */
24     public static int dividir(int a, int b) throws ArithmeticException {
25         return a / b;
26     }
27 }
28
29
```

Documentando un Enum

Al documentar un **Enum** en JavaDoc, es crucial describir su propósito y los valores que representa. Los Enums son tipos especiales de clases que limitan la elección de un valor a un conjunto predefinido de constantes, lo que los hace ideales para representar colecciones fijas y bien conocidas, como días de la semana, estados de un proceso o tipos de usuario.

Para cada constante dentro del Enum, se debe proporcionar una descripción clara de lo que significa y su uso previsto. Si el Enum tiene métodos o campos, también deben ser documentados siguiendo las mismas directrices que se aplicarían a una clase normal. Esto asegura que cualquier desarrollador que utilice el Enum comprenda rápidamente su estructura y cómo interactuar con sus valores y funcionalidades.

```
package main.java;

public enum MiEnum {
    VALOR1, // Descripción del primer valor
    VALOR2  // Descripción del segundo valor
}
```

Documentando una Interfaz

Al documentar una **Interfaz** en JavaDoc, el objetivo principal es definir claramente el contrato que dicha interfaz establece. Una interfaz especifica un conjunto de métodos que una clase debe implementar, sin proporcionar la implementación concreta. Por lo tanto, la documentación debe centrarse en describir el propósito de la interfaz, el comportamiento esperado de sus métodos y las responsabilidades que asume cualquier clase que la implemente. Esto es crucial para los desarrolladores que necesitan entender cómo usar la interfaz o cómo implementarla correctamente.

```
2
3 /**
4  * Descripción de la interfaz
5  * @author AsteriX
6  *
7  */
8 public interface MiInterfaz {
9
10     /**
11     *
12     * @param a
13     * @param b
14     */
15     public void func(int a, int b);
16 }
17
```

```
/**
 * Esta clase representa una calculadora simple que puede
realizar
 * operaciones aritméticas básicas.
 *
 * @author MiNombre
 * @version 1.0
 * @since 2023-05-15
 */
public class Calculadora {

    /**
     * Suma dos números enteros.
     *
     * @param a Primer número a sumar
     * @param b Segundo número a sumar
     * @return La suma de a y b
     */
    public static int sumar(int a, int b) {
        return a + b;
    }
}
```

Ejemplo de documentación JavaDoc

Etiquetas comunes de JavaDoc

@param

Describe un parámetro del método.

Ejemplo: @param nombre Nombre del usuario

@return

Describe el valor de retorno del método.

Ejemplo: @return true si la operación fue exitosa

@throws

Documenta una excepción que puede lanzar el método.

Ejemplo: @throws IllegalArgumentException si el argumento es nulo

@author

Indica el autor del código.

Ejemplo: @author Juan Pérez

Más etiquetas de JavaDoc

@version

Indica la versión del código.

Ejemplo: @version 1.2.3

@since

Indica desde qué versión está disponible.

Ejemplo: @since 2.0

@deprecated

Indica que el elemento está obsoleto.

Ejemplo: @deprecated Usar nuevoMetodo() en su lugar

@see

Proporciona una referencia a otro elemento.

Ejemplo: @see OtraClase#otroMetodo()



Generando documentación HTML

Pasos para generar JavaDoc

1. En Eclipse, selecciona Project > Generate Javadoc
2. Selecciona el proyecto o los paquetes para los que deseas generar documentación
3. Configura las opciones de generación (visibilidad, destino, etc.)
4. Haz clic en Finish para generar la documentación





Beneficios de usar

JavaDoc



Documentación estandarizada

Proporciona un formato consistente para documentar el código Java, facilitando su comprensión por otros desarrolladores.



Fácil navegación

La documentación HTML generada permite navegar fácilmente entre clases, métodos y paquetes relacionados.



Integración con IDEs

Los IDEs como Eclipse muestran la documentación JavaDoc al pasar el cursor sobre elementos del código, mejorando la productividad.



Buenas prácticas para JavaDoc

Ser conciso pero completo

Proporciona suficiente información para entender el propósito y uso del elemento, pero evita ser excesivamente detallado.

Mantener actualizada la documentación

Actualiza la documentación cuando cambies el código para evitar inconsistencias que puedan confundir a otros desarrolladores.

Documentar interfaces públicas

Prioriza la documentación de clases, métodos y campos públicos que serán utilizados por otros desarrolladores.

Usar HTML con moderación

JavaDoc permite usar etiquetas HTML para formatear el texto, pero úsalas con moderación para mantener la legibilidad.

```
/**
 * Representa un usuario del sistema con información básica.
 * Esta clase proporciona métodos para gestionar la
información
 * del usuario y validar sus credenciales.
 *
 * @author Equipo de Desarrollo
 * @version 2.1
 * @since 1.0
 */
public class Usuario {
    private String nombre;
    private String email;

    /**
     * Crea un nuevo usuario con nombre y email.
     *
     * @param nombre El nombre del usuario
     * @param email El email del usuario
     * @throws IllegalArgumentException Si el email no es válido
     */
    public Usuario(String nombre, String email) {
        // Implementación
    }
}
```

Ejemplo de documentación completa

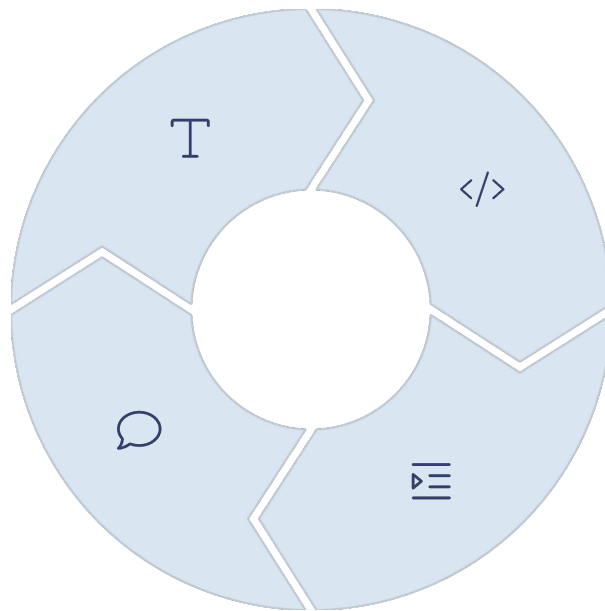
Resumen de estándares de codificación

Convención de nombres

Clases en CamelCase, métodos y variables en camelCase, constantes en MAYÚSCULAS_CON_GUIONES.

Documentación

Uso de JavaDoc para documentar clases, métodos y campos públicos.



Organización del código

Importaciones específicas y agrupadas por origen (JDK, terceros).

Formato y sangría

Sangría de 4 espacios, llaves de apertura en la misma línea que la declaración.

Resumen de aplicaciones de consola

Crear proyecto en Eclipse

Desarrollar una aplicación Java que interactúe con el usuario a través de la consola usando Scanner.

Exportar como JAR ejecutable

Usar la opción "Export > Runnable JAR file" para crear un archivo ejecutable.

Ejecutar desde terminal

Usar el comando "java -jar MiApp.jar" para ejecutar la aplicación desde la línea de comandos.

Resumen de depuración



Colocar breakpoints

Hacer clic en el margen izquierdo del editor para establecer puntos de interrupción.



Iniciar modo Debug

Usar la opción "Debug" en lugar de "Run" para iniciar la aplicación en modo de depuración.



Analizar variables

Examinar el estado de las variables y el flujo de ejecución en cada breakpoint.



Controlar ejecución

Usar F5 (Step Into), F6 (Step Over) y F8 (Resume) para controlar el flujo de ejecución.

Resumen de JavaDoc

Sintaxis básica

Comentarios que comienzan con `/**` y terminan con `*/`

Etiquetas principales

`@param`, `@return`, `@throws`, `@author`,
`@version`, `@since`, `@deprecated`, `@see`

Generación de documentación

Project > Generate Javadoc para crear documentación HTML navegable

Buenas prácticas

Ser conciso pero completo, documentar interfaces públicas, mantener actualizada la documentación

Ejercicio práctico: Estándares de codificación

Instrucciones

Revisa el siguiente código y corrige los problemas de convención de nombres y formato:

```
public class usuarios {  
    public static final int edad_minima=18;  
    private String Nombre;  
  
    public void Registrar(){  
        System.out.println("Usuario registrado");  
    }  
}
```

Solución

```
public class Usuario {  
    public static final int EDAD_MINIMA = 18;  
    private String nombre;  
  
    public void registrar() {  
        System.out.println("Usuario registrado");  
    }  
}
```

Ejercicio práctico: Aplicación de consola

Instrucciones

Crea una aplicación de consola simple que solicite al usuario su nombre y edad, y luego muestre un mensaje personalizado.

Solución

```
import java.util.Scanner;

public class Saludo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Ingrese su nombre:");
        String nombre = scanner.nextLine();

        System.out.println("Ingrese su edad:");
        int edad = scanner.nextInt();

        System.out.println("Hola " + nombre +
            ", tienes " + edad + " años.");

        scanner.close();
    }
}
```

Ejercicio práctico: Aplicación de consola

Ejercicio práctico: Depuración

Solución

Instrucciones

Identifica y corrige el error en el siguiente código utilizando las herramientas de depuración:

```
public class Calculadora {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 0;  
        int resultado = dividir(a, b);  
        System.out.println("Resultado: " + resultado);  
    }  
  
    public static int dividir(int a, int b) {  
        return a / b;  
    }  
}
```

```
public class Calculadora {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 0;  
        try {  
            int resultado = dividir(a, b);  
            System.out.println("Resultado: " + resultado);  
        } catch (ArithmeticException e) {  
            System.out.println("Error: División por cero");  
        }  
    }  
  
    public static int dividir(int a, int b) {  
        if (b == 0) {  
            throw new ArithmeticException("División por cero");  
        }  
        return a / b;  
    }  
}
```

Recursos adicionales

Documentación oficial de Java

La documentación oficial de Oracle proporciona guías detalladas sobre convenciones de código, JavaDoc y más.

<https://docs.oracle.com/en/java/javase/25/>

Guía de estilo de Google para Java

Una guía completa de estilo de codificación para Java utilizada por Google.

<https://google.github.io/styleguide/javaguide.html>

Tutorial de Eclipse

Tutoriales detallados sobre cómo usar Eclipse para desarrollo y depuración de Java.

<https://www.eclipse.org/documentation/>

Herramientas complementarias



Checkstyle

Una herramienta de análisis estático que ayuda a los programadores a escribir código Java que se adhiere a un estándar de codificación.



FindBugs

Una herramienta de análisis estático que busca posibles errores en el código Java.



PMD

Una herramienta de análisis de código fuente que encuentra problemas comunes de programación como variables no utilizadas, bloques catch vacíos, creación de objetos innecesarios, etc.

Consejos para mejorar tu código Java

Mantén las clases pequeñas y enfocadas

Cada clase debe tener una única responsabilidad. Si una clase hace demasiadas cosas, considera dividirla en clases más pequeñas.

Evita comentarios obvios

Los comentarios deben explicar el "por qué", no el "qué". El código debe ser lo suficientemente claro para entender qué hace sin necesidad de comentarios.

Usa nombres descriptivos

Los nombres de variables, métodos y clases deben ser descriptivos y revelar su intención. Evita abreviaturas confusas.

Maneja adecuadamente las excepciones

No ignores las excepciones. Captúralas y manéjalas apropiadamente, o declara que tu método las lanza.

Preguntas frecuentes

¿Es obligatorio seguir las convenciones de nombres en Java?

No es obligatorio, pero seguir las convenciones estándar hace que tu código sea más legible y profesional, facilitando la colaboración con otros desarrolladores.

¿Puedo ejecutar aplicaciones Java sin IDE?

Sí, puedes compilar y ejecutar aplicaciones Java desde la línea de comandos usando `javac` y `java`, o ejecutar archivos JAR con `java -jar`.

¿Cuándo debo usar JavaDoc?

Debes usar JavaDoc para documentar todas las clases, métodos y campos públicos que forman parte de la API que otros desarrolladores utilizarán.

Errores comunes a evitar

No cerrar recursos

Siempre cierra recursos como archivos, conexiones de base de datos y Scanner usando try-with-resources o en bloques finally.

Comparar strings con ==

Usa el método equals() para comparar el contenido de strings, no el operador ==, que compara referencias.

Ignorar excepciones

No dejes bloques catch vacíos. Siempre maneja las excepciones adecuadamente, al menos registrándolas.

Código duplicado

Evita copiar y pegar código. Si necesitas la misma funcionalidad en varios lugares, extráela a un método.

Próximos pasos en tu aprendizaje

Programación orientada a objetos avanzada

Profundiza en conceptos como herencia, polimorfismo, interfaces y clases abstractas.

Programación concurrente

Explora la programación multihilo y las utilidades de concurrencia de Java.

Colecciones y genéricos

Aprende a usar las estructuras de datos de Java como List, Set, Map y cómo trabajar con genéricos.

Desarrollo de aplicaciones con interfaz gráfica

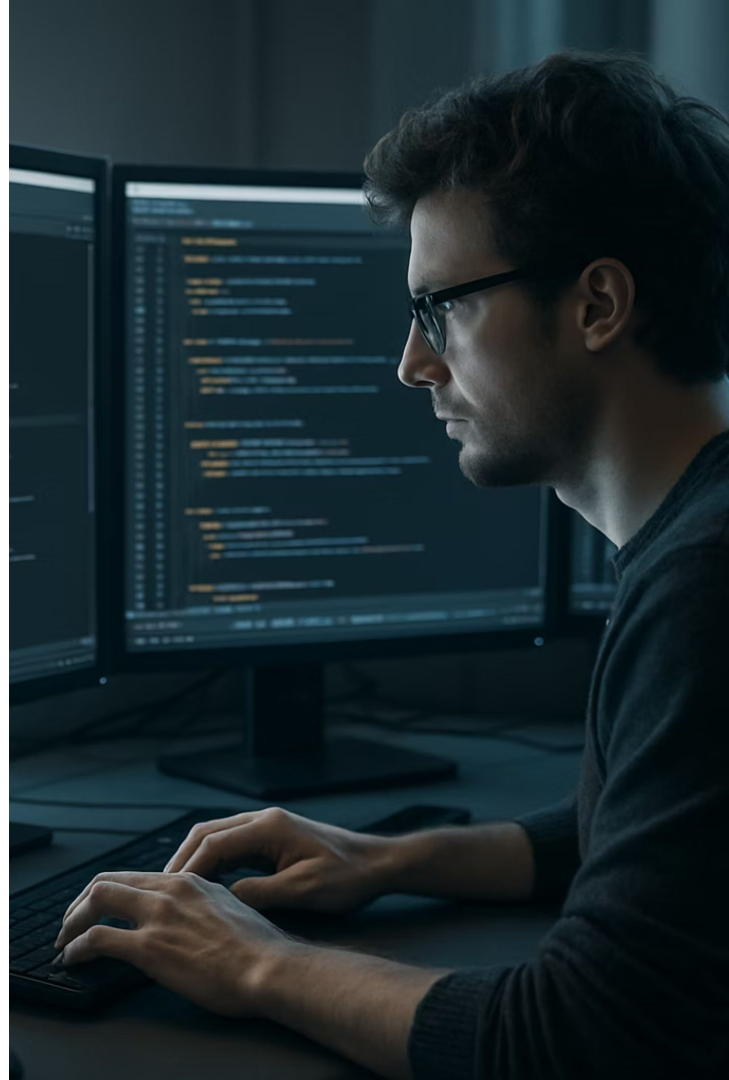
Aprende a crear aplicaciones con interfaces gráficas usando JavaFX o Swing.

Conclusión

En esta lección pudimos conocer y aprender los distintos estándares a la hora de escribir código en Java, estándares que nos ayudaran a poder destacar como buenos conocedores del lenguaje y poder estar en sintonía con otros desarrolladores.

Por otro lado, aprendimos cómo compilar y generar nuestro archivo ejecutable de Java, aplicación que podremos correr desde cualquier maquina que tenga instalado Java 25

Y finalmente, aprendimos a depurar nuestro código para buscar errores de la forma más profesional posible y cómo documentar nuestra app, con el fin de hacerla más legible y mantenible.



Live Coding

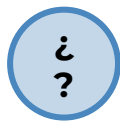
¿En qué consistirá la Demo?

Crearemos una clase *Saludo*, documentaremos sus métodos y atributos usando *JavaDoc*, y generaremos la documentación en *HTML* desde *Eclipse*.

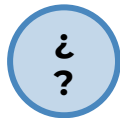
1. Crear clase *Saludo*.
2. Agregar atributos y método principal.
3. Usar *Scanner* para leer datos.
4. Agregar documentación *JavaDoc*.
5. Usar etiquetas como *@param*, *@return*.
6. Generar documentación desde *Eclipse*.
7. Mostrar cómo navegar el *HTML*.
8. Ejecutar desde consola y verificar salida.

Tiempo: 20 minutos

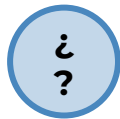
#Momentode Preguntas...



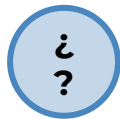
¿Qué ventajas tiene documentar con JavaDoc en lugar de comentarios comunes?



¿Qué etiquetas de JavaDoc considerarás esenciales en cualquier proyecto?



¿Cómo ayuda la depuración a mejorar la calidad del software?



¿Cuál sería el siguiente paso para mejorar la app de saludo?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Verificador de números pares

Verificador de números pares

Contexto: 🙌

Una empresa necesita una herramienta simple que permita verificar si un número ingresado por el usuario es par o impar. Este ejercicio sirve como base para entrenar a nuevos programadores en validación, documentación profesional con JavaDoc y depuración paso a paso con Eclipse

Consigna: 📝

Desarrolla una aplicación de consola en Java llamada VerificadorPar, que:

- Solicite al usuario que ingrese un número entero.
- Valide que el número no sea negativo.
- Determine si el número es par o impar.
- Documente toda la clase y métodos usando JavaDoc.
- Coloque puntos estratégicos para practicar la depuración con Eclipse.

Tiempo 🕒: 45 minutos

Verificador de números pares

Paso a paso:

1. Crear el proyecto VerificadorPar en Eclipse.
2. Declarar la clase principal con método main.
3. Implementar un método para leer el número (leerNumero).
4. Crear un método esPar(int numero) que devuelva true o false.
5. Mostrar el resultado al usuario.
6. Lanzar excepción si el número es negativo.
7. Agregar JavaDoc a todos los métodos y clases.
8. Colocar breakpoints en la lectura del número, validación y resultado.
9. Ejecutar en modo Debug para analizar el flujo y el estado de variables.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Aprendimos a documentar código con JavaDoc**
- ✓ **Usamos etiquetas clave como @param y @return**
- ✓ **Aplicamos estándares de formato y organización**
- ✓ **Depuramos un error con Eclipse Debug**
- ✓ **Creamos una app de consola correctamente documentada**
- ✓ **Generamos documentación HTML**
- ✓ **Conocimos herramientas de análisis como Checkstyle**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

